Capstone Project

Instructor: Shajan Dasan

LOGISTICS & SUPPORT

- Project Choice & Complexity: Choose one of the 6 listed projects, or bring your own. You must ensure your project is complex enough.
- Duration: Four weeks to work on this project.
- Submission
    - Zip code along with a README and upload via a Google Form link OR create a public repo on github (you will receive email).
    - Do not add your api keys / .env in the uploaded version
    - Demo video (5 - 10 mins)
- Support Sessions: Two Thursday sessions and one Sunday session are available for questions and work-in-progress demos.
- Uplevel shared Shared with students
    - New link Uplevel shared

PROJECTS OVERVIEW

Project Name	Brief Description	Technical Focus
Finance Assistant	Personalized financial education and guidance using multi-agent systems.	Multi-Agent Architecture (LangGraph/CrewAI), RAG/Vector DB, External API integration.
Content Marketing	SEO-optimized content generation across multiple formats for small businesses.	Multi-Agent Architecture (LangGraph), Specialized Agents (6), Content Enhancement Pipeline, Redundancy.

Project Name	Brief Description	Technical Focus
Call Center Intel	Automated QA analysis and speaker labeling for call center operations.	LangGraph State Machine (7 stages), PII Redaction/Security, Deterministic Scoring, Cost Control.
Customer Support	Secure chat support for music stores with anti-hallucination safeguards.	Multi-Agent Orchestration, Anti-Hallucination Safeguards, Secure Identity Verification.
IT & HR Ops	First-line help desk with RAG-based knowledge retrieval and ticketing.	Confidence Handling (RAG), API Integration (external ticketing), Orchestration (Query, Create, Status).
Multi-Agent Creative Studio	Multi-agent system for Instagram campaign generation with research, copy, design, review, and planning agents.	Distributed Multi-Agent Architecture (A2A), Google Cloud Run/Agent Engine, Multimodal Image Generation, Revision Loop.

Pre-Class Setup Recommendations

- Make sure you have OpenAI Keys - check email / work with IK Ops if you are missing it.
- Signups should happen in week 1. Alpha Vantage takes up to 24 hours to issue keys; Jira/Asana/Notion auth flows can take a full session to debug. "I can't get my key working" is the #1 reason students miss milestones.
- Projects 3 and 4 link to existing GitHub repos - do not look at these until you've finished your own first pass. The learning happens as you build.
- Project 3 needs Python 3.11 or 3.12 (not 3.13). Project 4 needs Python 3.12 specifically (3.13 breaks Gradio).
- Project 4 has the lowest setup friction for students who just want to start coding — only the program OpenAI key is required.

- Project 5 has the most creative latitude — framework-agnostic, no reference code, but the ticketing API is the wildcard.

Project 1 — Finnie: AI Finance Assistant

Customer Problem

Most adults want to invest but are blocked by jargon, paralyzed by choices, and afraid of making mistakes. Human financial advisors don't scale to beginners — they're expensive and have account minimums. Generic robo-advisors push products without teaching the underlying concepts. The result: millions of people leave money in low-yield savings accounts instead of building wealth, because the on-ramp into investing is intimidating and lonely.

Proposed Solution

A multi-agent conversational tutor that meets beginners where they are. Six specialist agents collaborate behind a single chat interface:

- Finance Q&A Agent — explains concepts like compound interest, ETFs vs mutual funds.
- Portfolio Analysis Agent — analyzes uploaded holdings for diversification and risk.
- Market Analysis Agent — pulls live quotes and explains what the numbers mean.
- Goal Planning Agent — calculates how much to save monthly to hit a target.
- News Synthesizer Agent — summarizes current financial news with sources.
- Tax Education Agent — explains 401(k) vs IRA vs Roth without giving tax advice.

A LangGraph router classifies each query and dispatches it to the right agent. RAG over a 50-100 article knowledge base grounds answers in verified content. A Streamlit UI ties it together with tabs for Chat, Portfolio, Markets, Goals, and

Knowledge. The system explicitly distinguishes education from advice and adds disclaimers — a real regulatory concern in fintech.

Tech Stack

LangGraph, LangChain, OpenAI (or Google Gemini as a free alternative), FAISS vector store, sentence-transformers (all-MiniLM-L6-v2) for embeddings, Streamlit UI, async Python throughout.

External APIs & Services Required

Service	Purpose	Notes
OpenAI API	LLM for all agents	Provided by the program
Alpha Vantage	Live market data	Free: 25 req/day, 5 req/min — students sign up themselves
SerpApi or Tavily	News search	Free: SerpApi 100/mo, Tavily 1,000/mo — students sign up themselves

Heads up: Alpha Vantage's 25-req/day cap is tight. Students must build aggressive caching (30-min TTL) and mock data fallbacks early — this is the #1 demo killer.

Difficulty: Medium
A complete reference implementation is provided. Good starting project for students who want to learn multi-agent patterns and RAG without too much infrastructure overhead.

Project 2 — ContentBlitz: Content Marketing Agent

Customer Problem

Solo creators and small marketing teams have to produce content across formats — blog posts, LinkedIn posts, social images — every week to stay visible. Each format has different rules: SEO keyword density for blogs, 1300-character hooks for LinkedIn, prompt craft for images. Quality research takes hours per piece. Big companies have dedicated content teams and design departments; everyone else either burns out or publishes mediocre output that doesn't convert.

Proposed Solution

A multi-agent content factory where each agent specializes in one content type, coordinated by a smart LLM-based router. Six agents:

- Query Handler — routes the request (research? blog? LinkedIn? image?).
- Deep Research Agent — generates multiple strategic search queries, synthesizes findings with source citations.
- SEO Blog Writer — produces long-form content with keyword density, meta description, H1/H2/H3 structure.
- LinkedIn Writer — 1300-1600 char posts with hook, story, CTA, and 8-12 hashtags.
- Image Generator — enhances user prompts, generated via DALL-E with fallback chain (gpt-image-1 → DALL-E 3 → DALL-E 2).
- Content Strategist — formats research into readable reports when no specific format was requested.

The router does both initial routing (what does the user want?) and post-research routing (now that we've researched, are we writing a blog, a LinkedIn post, or just a research summary?). Built on LangGraph with langgraph dev for local iteration and LangGraph Cloud-ready deployment.

Tech Stack

LangGraph, LangChain, OpenAI GPT-4 Turbo, DALL-E for image generation, SerpAPI for web research, Pillow + matplotlib for image handling.

External APIs & Services Required

Service	Purpose	Notes
OpenAI API	LLM + DALL-E image generation	Provided by the program
SerpAPI	Web research	Free: 100 searches/month — students sign up themselves

Heads up: The newer gpt-image-1 model requires a verified OpenAI organization. Confirm whether the program's shared OpenAI keys have org verification — if not, the fallback chain to DALL-E 3 → DALL-E 2 will be the default path. Either works for the project requirements.

Difficulty: Medium
Single-file reference implementation provided. Good fit for students interested in content creation, prompt engineering, and multi-modal AI (text + images).

Project 3 — Call Center Intelligence System

Customer Problem

A mid-size call center handles roughly 5,000 calls per day. QA teams manually review fewer than 5% of them, taking about 15 minutes per review. This creates three compounding failures: 95% of calls receive zero quality review (coaching opportunities lost), human reviewers only agree on the same call 40-60% of the time (inconsistent and unfair scoring), and reviews surface problems days after

the call (too late to coach or correct). In regulated industries like finance and healthcare, this is also a compliance liability — there's no audit trail and no way to prove required disclosures were made.

Proposed Solution

A production-grade 7-stage LangGraph pipeline that processes call audio end-to-end:

1. Intake — magic-byte format validation (not file extension), 50 MB and 60-minute limits.
2. Transcription — faster-whisper with int8 quantization, speaker diarization, SHA-256 caching so identical audio returns instantly.
3. Injection Detection — 22 regex patterns block adversarial audio before any LLM sees the transcript.
4. PII Redaction — strips SSN, credit card, email, and phone before the LLM call (GDPR/CCPA safety).
5. Summarization — structured Pydantic output: call purpose, action items, sentiment trajectory, entities.
6. QA Scoring — five weighted dimensions (Professionalism 15%, Empathy 20%, Resolution 30%, Compliance 20%, Clarity 15%); the overall score is always recomputed deterministically in Python — the LLM's score is discarded as a guardrail against scoring drift.
7. Report — PDF and JSON generation via ReportLab, SQLite persistence, append-only audit log.

A Gradio app with two tabs (Analyze Call, Observability dashboard) and 100+ tests across unit, integration, and security suites. A multi-provider LLM factory makes switching between OpenAI, Gemini, and Groq a single environment variable change — no code edits required, which is itself a graded requirement.

Tech Stack

Python 3.11+, LangGraph, faster-whisper (CTranslate2 backend), Pydantic v2, SQLite + SQLAlchemy, ReportLab for PDFs, Gradio 5.x, pytest, ruff. Optional LangSmith for tracing.

External APIs & Services Required

- Service: OpenAI API (GPT-4o) | Purpose: Primary LLM for analysis | Notes: Provided by the program — recommended default.
- Service: Google Gemini 2.0 Flash | Purpose: Alternative LLM | Notes: Free 1,500 req/day — students must still wire up the provider factory for full marks.
- Service: Groq Llama 3.3 70B | Purpose: Alternative LLM | Notes: Free 30 req/min — same as above.
- Service: LangSmith | Purpose: Tracing/observability | Notes: Free tier, optional.

Reference implementation:
https://github.com/ANI-IN/Call-Center-Intelligence-System — students should not look at this until after completing their own first pass.

Heads up: This is the most demanding project. faster-whisper downloads model weights on first run (1-3 GB depending on size). On CPU laptops, students must use WHISPER_MODEL_SIZE=tiny during development — large-v3 takes 25+ minutes per call on CPU. Python 3.11 or 3.12 only, not 3.13. Also: students still need to implement the multi-provider factory (it's worth 20% of the grade) even though they'll likely run with OpenAI for the demo.

Difficulty: Hard
Recommended for students with strong Python fundamentals who want production engineering experience: typed state machines, security layers, deterministic scoring, multi-provider abstraction, and 100+ tests.

Project 4 — Multi-Agent Customer Support (Digital Music Store)

Customer Problem

A digital music store has 59 customers, 3,500 tracks, and handles support manually. Customers email or call to ask questions like "What rock albums do you

have?" or "What was on my last invoice?" Support agents spend their days on repetitive lookups instead of solving real problems. The company wants to automate this with an AI assistant but has hard constraints: the bot must never fabricate album names or invoice amounts, customers must verify their identity before accessing account data, and the experience should improve over time by remembering preferences.

Proposed Solution

A supervisor-based multi-agent system over the Chinook sample database with two specialists:

- Supervisor — routes queries to specialists, rejects off-topic questions directly, handles mixed queries by calling both agents sequentially.
- Music Catalog Agent (5 tools) — albums by artist, tracks by artist, songs by genre (with artist diversity), song search by title, track details by ID.
- Invoice Agent (4 tools) — customer invoices sorted by date, purchased tracks sorted by price, support rep lookup, invoice line items.

Three engineering patterns make this project distinctive:

- Identity verification with human-in-the-loop interrupt — the graph pauses, asks for customer ID / email / phone, looks up in the database with phone normalization and case-insensitive email matching, then injects the verified customer_id via a SystemMessage. Downstream agents never extract IDs from free-text user messages.
- Preference memory with set-union merge — "I love jazz" gets saved as a preference; "Do you have jazz?" does not. New preferences merge with existing ones; nothing is ever overwritten. Memory persists across conversation turns and is scoped per customer.
- Parameterized SQL only — every database query uses SQLAlchemy text() with parameter bindings. No f-strings or string concatenation. This defends against SQL injection.

Every agent prompt includes explicit anti-hallucination grounding rules: quote exact numbers, say "no results" honestly, defer when out of scope, never answer

from training data. A Gradio chat UI uses per-session thread IDs so browser tabs don't bleed conversations into each other.

Tech Stack

Python 3.12 (not 3.13), LangGraph, LangChain, langgraph-supervisor for routing, SQLAlchemy with in-memory SQLite, Pydantic v2 for structured LLM outputs, Gradio with Blocks API, pytest. The Chinook database auto-downloads from GitHub on first run.

External APIs & Services Required

Service	Purpose	Notes
OpenAI API	LLM for all agents	Provided by the program
Chinook database	Source data	Free, auto-downloaded — no key needed

Reference implementation: https://github.com/ANI-IN/Multi-Agent-Customer-Support — do not look until after first pass.

Heads up: Python 3.12 only. Python 3.13 removed the audioop module that Gradio depends on transitively. This causes an ImportError at startup and will burn hours of debugging time. Call it out in the kickoff session.

Difficulty: Medium-Hard
Good fit for students who want to learn supervisor patterns, human-in-the-loop flows, and production data safety practices (SQL parameterization, identity injection, anti-hallucination). The lowest-friction project from an API setup perspective — only the program-provided OpenAI key is needed.

Project 5 — SmartDesk AI: IT & HR Help Desk Agent

Customer Problem

IT and HR teams in most organizations are buried in repetitive tickets — password resets, leave policies, VPN setup, reimbursement questions, onboarding checklists. The answers are usually documented in internal wikis and policy PDFs, but finding the right document is harder than asking a human. So employees raise tickets, wait days for a reply, and the support team spends their time pasting links instead of solving real problems that need human judgment.

Proposed Solution

A conversational agent that sits between employees and the support team with three capabilities:

1. RAG-based answer — searches a curated 30-50 entry IT/HR knowledge base using dense vectors, sparse BM25, or hybrid search. Grounds answers in retrieved chunks. Says "I don't know" honestly when retrieval confidence is low.
2. Ticket creation (write operation) — when the agent can't answer, it offers to file a ticket. Collects employee email, issue summary, description, category (IT vs HR), and optional priority. Shows a human-in-the-loop confirmation summary before submitting. Returns the ticket ID or link.
3. Ticket status check (read operation) — looks up tickets by employee email via the external ticketing API. Handles multiple-ticket disambiguation (lists tickets, lets the employee pick one). Reports status and any team comments.

The most interesting design decision is the escalation logic: how does the agent know when it doesn't know? Students choose from retrieval-score thresholds, LLM self-assessment via system prompt, no-retrieval fallback, or some combination. The knowledge base must include deliberate gaps so the escalation

path is testable — if the agent always finds an answer, you can't tell whether it's working.

The project is deliberately framework-agnostic. Students can use LangGraph, LangChain, Google ADK, CrewAI, AutoGen, or raw function calling. Knowledge base content can come from Hugging Face datasets (strova-ai/hr-policies-qa-dataset, IT helpdesk datasets), LLM-generated synthetic policies, or a mix. The marking scheme rewards realism, breadth, and deliberate testability.

Tech Stack (Student's Choice)

- Agent framework: LangGraph, LangChain, Google ADK, CrewAI, AutoGen, Semantic Kernel, or raw function-calling.
- LLM: OpenAI (recommended — provided), Anthropic, Gemini, Groq, or local Ollama.
- Embeddings: OpenAI (provided), Cohere, Google, or Hugging Face sentence-transformers (free, local).
- Vector DB: FAISS, ChromaDB, Pinecone, Qdrant, Weaviate, pgvector.
- UI: Streamlit, Gradio, Chainlit, FastAPI, or CLI.

External APIs & Services Required

- Service: OpenAI API | Purpose: LLM + embeddings | Notes: Provided by the program.
- Service: Ticketing platform | Purpose: Write + read tickets | Notes: Jira / Asana / Notion / Linear / GitHub Issues — all have free tiers with API access; student picks one and signs up.
- Service: LangSmith / Ragas / DeepEval | Purpose: Evaluation pipeline | Notes: Free tiers, bonus marks only.

Heads up: The ticketing API integration is where most students will struggle. Each platform has its own auth flow — Jira uses API tokens, Notion uses integration tokens, GitHub uses Personal Access Tokens. Pick one platform to walk through in office hours. The rubric explicitly allows mocked API calls for partial credit if students can't get live integration working.

Difficulty: Medium
This is the safest project for students who want creative latitude — framework-agnostic, no reference code to peek at, lowest external-dependency risk. The challenge is designing good escalation logic and a knowledge base with realistic gaps.

Project 6 Multi-Agent Creative Studio

Customer Problem

Producing a social media campaign is a slow, manual relay race. Marketers brief strategists, who research audiences. Copywriters draft captions. Designers create visuals. Reviewers send revisions back and forth. Each handoff creates queues and bottlenecks. "Small" campaigns take days, scaling is impossible, and context is lost between roles.

Proposed Solution

A distributed multi-agent system where a "Creative Director" orchestrator manages five specialist agents:

- Brand Strategist - researches audience, competitors, and trends.
- Copywriter - produces three captioned variations (inspirational, educational, community-focused).
- Designer - generates images using Gemini and stores them in Cloud Storage.
- Critic - scores output (1-10) and enforces a quality gate (APPROVED/NEEDS_REVISION).
- Project Manager - creates timelines and optionally syncs to Notion via MCP.

The Creative Director handles the revision loop: if the Critic rejects work, the orchestrator routes feedback to the Copywriter or Designer, then re-checks, with no human in the loop.

Tech Stack

Python 3.11+, LangGraph, Google Agent Development Kit (ADK), Google Cloud Run (specialists), Vertex AI Agent Engine (orchestrator), Google Cloud Storage (images), Notion MCP (optional).

External APIs & Services Required

- Vertex AI (Gemini) — Provided by the program.
- Google Cloud Platform — Standard services (Cloud Run, Storage, etc.).

Difficulty: Hard
A distributed system requiring independent services, A2A communication, cloud deployment, and robust error handling. Excellent for students wanting to build production-grade agent systems.

Quick Comparison

Project	Difficulty	Extra Services Beyond OpenAI	Framework	Reference Code
Finnie	Medium	Alpha Vantage + SerpApi/Tavily	LangGraph + Streamlit	Full repo provided
ContentBlitz	Medium	SerpAPI	LangGraph	Single-file reference
Call Center	Hard	None required (Gemini/Groq optional)	LangGraph + Gradio	External GitHub link
Music Store	Medium-Hard	None	LangGraph + Gradio	External GitHub link

Project	Difficulty	Extra Services Beyond OpenAI	Framework	Reference Code
SmartDesk	Medium	Ticketing platform (Jira/Notion/etc.)	Student's choice	Spec only
Multi-Agent Creative Studio	Hard	Vertex AI / GCP	LangGraph + ADK	Tutorial provided

References & Resources

Agent Frameworks & Orchestration

- LangGraph — LangGraph
- LangGraph Cloud Deployment — LangGraph Cloud Deployment
- LangChain (Python) — LangChain (Python)
- LangChain OpenAI Integration — LangChain OpenAI Integration
- langgraph-supervisor — langgraph-supervisor
- CrewAI — CrewAI
- AutoGen (Microsoft) — AutoGen (Microsoft)
- A2A Protocol — A2A Protocol
- Google Agent Development Kit (ADK) — Google ADK
- Semantic Kernel — Semantic Kernel

LLM Providers

- OpenAI Platform — OpenAI Platform
- OpenAI API Reference — OpenAI API Reference
- OpenAI DALL-E / Images API — OpenAI DALL-E / Images API
- Anthropic Claude API — Anthropic Claude API
- Google AI Studio (Gemini) — Google AI Studio (Gemini)
- Google Gemini API Docs — Google Gemini API Docs
- Groq API — Groq API

- Ollama (local LLMs) — Ollama

Speech-to-Text

- faster-whisper — faster-whisper
- OpenAI Whisper (original) — OpenAI Whisper
- CTranslate2 — CTranslate2

Vector Databases & Embeddings

- FAISS — FAISS
- ChromaDB — ChromaDB
- Pinecone — Pinecone
- Qdrant — Qdrant
- Weaviate — Weaviate
- pgvector — pgvector
- sentence-transformers — sentence-transformers
- all-MiniLM-L6-v2 model card — all-MiniLM-L6-v2 model card
- OpenAI Embeddings — OpenAI Embeddings

Web Search & Research APIs

- SerpApi — SerpApi
- Tavily AI — Tavily AI
- Perplexity Sonar API — Perplexity Sonar API

Market Data (Project 1)

- Alpha Vantage — Alpha Vantage
- yfinance — yfinance

Image Generation (Project 2)

- OpenAI Image Generation Guide — OpenAI Image Generation Guide
- Stability AI — Stability AI
- Pillow (PIL) — Pillow (PIL)

Web UI Frameworks

- Streamlit — Streamlit
- Gradio — Gradio
- Chainlit — Chainlit
- FastAPI — FastAPI

Ticketing Platforms (Project 5)

- Jira Cloud REST API — Jira Cloud REST API
- Asana API — Asana API
- Notion API — Notion API
- Linear API — Linear API
- GitHub Issues API — GitHub Issues API
- Trello REST API — Trello REST API

Data Validation & Modeling

- Pydantic v2 — Pydantic v2

Databases & ORMs

- SQLAlchemy — SQLAlchemy
- SQLite — SQLite
- Chinook Sample Database (Project 4) — Chinook Sample Database (Project 4)

Document & Report Generation

- ReportLab (PDF) — ReportLab (PDF)
- mutagen (audio metadata) — mutagen (audio metadata)

Observability & Evaluation

- LangSmith — LangSmith
- Ragas (RAG eval) — Ragas (RAG eval)
- DeepEval — DeepEval
- Weights & Biases — Weights & Biases

- Helicone — Helicone

Knowledge Base Sources (Project 5)

- Hugging Face Datasets — Hugging Face Datasets
- strova-ai/hr-policies-qa-dataset — strova-ai/hr-policies-qa-dataset
- Kaggle Datasets — Kaggle Datasets

Python Tooling

- python-dotenv — python-dotenv
- pytest — pytest
- ruff — ruff
- uv (fast package installer) — uv

Domain References (Finnie — Project 1)

- Investopedia — Investopedia
- Bogleheads wiki — Bogleheads wiki
- SEC Investor.gov — SEC Investor.gov

Domain References (ContentBlitz — Project 2)

- Moz SEO Beginner's Guide — Moz SEO Beginner's Guide
- HubSpot Content Marketing — HubSpot Content Marketing
- Google E-E-A-T Guidelines — Google E-E-A-T Guidelines

Compliance & Security (Projects 3 & 4)

- GDPR Overview — GDPR Overview
- CCPA Overview — CCPA Overview
- OWASP LLM Top 10 — OWASP LLM Top 10

Reference Implementations

- Call Center Intelligence System (Project 3) — Call Center Intelligence System (Project 3)

- Multi-Agent Customer Support (Digital Music Store) — Multi-Agent Customer Support (Digital Music Store)

Cloud Infrastructure

- Google Cloud Run — Google Cloud Run
- Vertex AI Agent Engine — Vertex AI Agent Engine
- Google Cloud Storage — Google Cloud Storage
- Secret Manager — Secret Manager
- Artifact Registry — Artifact Registry
- Model Context Protocol (MCP) — Model Context Protocol (MCP)
- Multi-Agent Creative Studio (Project 6) — Multi-Agent Creative Studio (Project 6)
- Multi-Agent Creative Studio Reference Walkthrough — Multi-Agent Creative Studio Reference Walkthrough

Project GitHub Solutions

- **Project 1 — Finnie: AI Finance Assistant**: No direct GitHub repository link provided in document.
- **Project 2 — ContentBlitz: Content Marketing Agent**: No direct GitHub repository link provided in document.
- **Project 3 — Call Center Intelligence System**: https://github.com/ANI-IN/Call-Center-Intelligence-System
- **Project 4 — Multi-Agent Customer Support (Digital Music Store)**: https://github.com/ANI-IN/Multi-Agent-Customer-Support
- **Project 5 — SmartDesk AI: IT & HR Help Desk Agent**: No direct GitHub repository link provided in document.
- **Project 6 — Multi-Agent Creative Studio**: https://github.com/ANI-IN/Multi-Agent-Creative-Studio